

02 | LLM Architecture - Week 06 | LLM Training – Summary

01.04.2026

The class focuses on the **training phases of Large Language Models (LLMs)**, specifically diving into pre-training and fine-tuning, with a brief mention of alignment.

LLM Training Phases

LLM training is typically divided into three main phases:

1. **Pre-training:** This is the initial phase where an LLM is trained from scratch on a massive dataset. It involves building the base LLM.
2. **Fine-tuning:** This phase adapts an existing, pre-trained LLM for specific tasks or domains, often using smaller datasets than pre-training.
3. **Alignment:** This final phase involves methods like reinforcement learning with human feedback (RLHF) to align LLMs with safety policies and ethical behavior. This is a dedicated module later in the AI engineering training program.

Pre-training

Pre-training involves training transformer-based LLMs on enormous amounts of data to give them vast knowledge, which enables their broad capabilities.

- **LLM Architectures:** There are three main transformer-based architectures for LLMs:

- **Decoder-only models:** These are causal, auto-regressive, left-to-right LLMs that excel at generating text by predicting one token at a time. Examples include GPT, Claude, Llama, and Mixtra.

- **Encoder-only models:** These are masked language models (MLMs), like BERT, trained to predict masked tokens based on surrounding words. They are typically fine-tuned for classification tasks. BERT uses special CLS and separator tokens for full sentence representation and sentence differentiation, respectively.

- **Encoder-decoder models:** These are sequence-to-sequence models popular for machine translation and speech recognition.

- **Data for Training:** LLMs are pre-trained on vast datasets, including academic papers (PubMed, arXiv), web data (PIRCC, Stock Exchange, OpenWeb Text, Wikipedia), books

(Bibliotique), coding platforms (GitHub), mathematics (DMATH), and dialogue conversations.

- **Challenges with Data:** Scraping web data for pre-training presents several issues:

- **Copyright:** Much online text is copyrighted, posing an open legal problem, especially for generative AI.

- **Data Consent:** Website owners may opt out of having their sites crawled.

- **Privacy:** Websites can contain private information (phone numbers, IP addresses, emails), which can inadvertently end up in the LLM.

- **Data Filtering:** Data undergoes filtering for quality and safety:

- **Quality:** Deduplication is crucial to avoid training on the same input multiple times.

- **Safety:** Filtering toxic and adult content is performed to ensure appropriate LLM behavior.

- **Bias:** Biases in data are handled, with ethical considerations varying by country.

Ethical committees are often involved in shaping the LLM's persona.

- **Loss Functions:**

- **Cross-Entropy Loss:** This is the common loss function used across LLM training. The goal is to assign a high probability to the true token, and the loss is high if a low probability is assigned.

- **Self-Supervised Setup:** This approach uses the next or missing word as the label.

- **BERT's Loss Functions:** BERT uses two loss functions concurrently:

1. **Masked Language Model (MLM):** Predicts masked tokens. BERT might sometimes replace a token instead of masking it to add noise.

2. **Next Sentence Prediction (NSP):** A binary classification task to determine if two sentences follow each other. This helps the model understand sentence relationships for tasks like entailment. The combined loss is the sum of these two components.

- **GPT's Loss Function (Causal LLMs):** GPT models use "teacher forcing." At each step, the model predicts the next token. Regardless of its prediction, the *correct* next token is used as input for the subsequent prediction, minimizing error through gradient descent.

- **Computational Effort and Scaling Laws:**

- Training large LLMs like GPT-5.5 requires immense computational resources, potentially involving trillions of parameters, tens of thousands of GPUs, and costing hundreds of millions of dollars over several months.
- **Scaling Laws** developed by OpenAI help predict model behavior (loss) as a function of the number of parameters, dataset size, and compute budget. These relationships show an exponentially decaying loss as compute, data, or parameters increase, indicating diminishing returns but significant performance gains from scale. Compute refers to floating-point operations, not just training time. Early stopping remains important for optimal performance.

Fine-tuning

Fine-tuning adapts a pre-trained model to new data, specializing it for a domain or task.

- **Purpose:** It's considered when an LLM needs to perform well on domains or languages underrepresented in its pre-training data, or for specific tasks. While Retrieval-Augmented Generation (RAG) can address many issues by grounding models in external knowledge, fine-tuning is valuable for shaping core behavior, improving consistency, reducing inference costs, and optimizing performance beyond what RAG offers.

- **Fine-tuning vs. In-context Learning:** Fine-tuning produces a *new model* by changing its parameters, unlike in-context learning (or few-shot learning), which is a form of prompt engineering on an existing model and doesn't change its underlying parameters.

- **Types of Fine-tuning:**

- **Full Parameter Tuning:** All parameters of the pre-trained model are updated during fine-tuning. This typically uses an instruction dataset of input-answer pairs, sometimes with context. Task-specific fine-tuning (e.g., summarization, translation) allows the model to learn specific nuances and vocabulary.

- **Challenges:** Full parameter fine-tuning is computationally very expensive, requiring significant processing power, memory, and time, similar to pre-training for each epoch. It also risks "catastrophic forgetting," where the model might lose its general knowledge when trained too narrowly on a specific task. Multi-task fine-tuning can mitigate this, but requires more data and resources.

◦ **Parameter-Efficient Fine-tuning (PEFT)**: This approach updates only a *subset* of the model's parameters, making fine-tuning more efficient and less prone to catastrophic forgetting. Common PEFT methods include:

- **Additive**: New layers are added on top of a frozen pre-trained model, and only these new layers are trained.

- **Adapters**: Layers are inserted *within* the frozen model, and only the adapter layers are trained.

- **Soft Prompting**: Manipulates the input by fine-tuning the embedding layer.

- **Selective**: Chooses specific subsets of parameters within the full network to fine-tune.

- **LoRA (Low-Rank Adaptation)**: The most common PEFT method for transformer-based LLMs.

- **Intuition**: Any matrix W can be decomposed into two lower-rank matrices, B and A ($W = BA$). This reduces the number of parameters needed to represent the update.

- **Mechanism**: Instead of directly updating the dense weight matrices (like WQ , WK , WV) during fine-tuning, LoRA *freezes* these original weights (W_0) and updates a low-rank approximation (ΔW) alongside them. This ΔW is itself decomposed into two smaller matrices, A and B , which are then trained. A is initialized randomly, and B is initialized to zero to preserve the original output at the start. During the forward pass, the input is multiplied by $W_0 + BA$; during backpropagation, only A and B are updated. The rank (r) of A and B is typically very small (e.g., 1 or 2), significantly reducing the number of trainable parameters.

- **Advantages**: LoRA is much faster, requires less GPU memory, is more cost-efficient, is less prone to catastrophic forgetting, and can even outperform full fine-tuning.

- **QLoRA (Quantized LoRA)**: An extended version of LoRA that further reduces resource needs by quantizing the precision of network parameters, typically to 4-bit.

- **Quantization**: Involves mapping numbers from a larger range (e.g., FP32) to a smaller range (e.g., Int8 or 4-bit) by splitting the range into "buckets". This saves bits but results in a loss of information.

- **Handling Large Matrices:** To prevent outliers from skewing quantization constants, large matrices are flattened and divided into blocks. Each block has its own scaling constant (CI), which is then used to quantize the values within that block.

- **Double Quantization:** To further save memory, these block-specific scaling constants (CI) are also quantized, which is called double quantization.

- **How it works:** The parameters (A and B) learned in LoRA are done in an integer space, and then mapped back to FP32 to be added to the original frozen W matrix during the forward pass. This results in huge savings in computational effort, money, and time.

The lecture concludes by emphasizing the importance of these three training steps and announces a homework assignment involving fine-tuning a model with LoRA.